OOPs (Object-Oriented Programming)

Programming Paradigm

A **programming paradigm** is a **method or approach to programming** that guides how you write and structure your code.

It provides:

- a specific **style of thinking**
- rules for **organizing code**
- techniques for solving problems
- patterns of **program control flow**

Common Types of Programming Paradigms

1. **Procedural Paradigm** - C, Python, Pascal
2. **Object-Oriented Paradigm** - Java, Python, C++
3. **Functional Paradigm** - Python (partially)

Procedural Oriented Approach

- A complex program is divided into small, independent, and reusable parts called **functions**.
- Each function is written so it can easily work with other functions in the program.
- You can **send data to a function** using arguments, and the function can **send back a result** to the place where it was called.

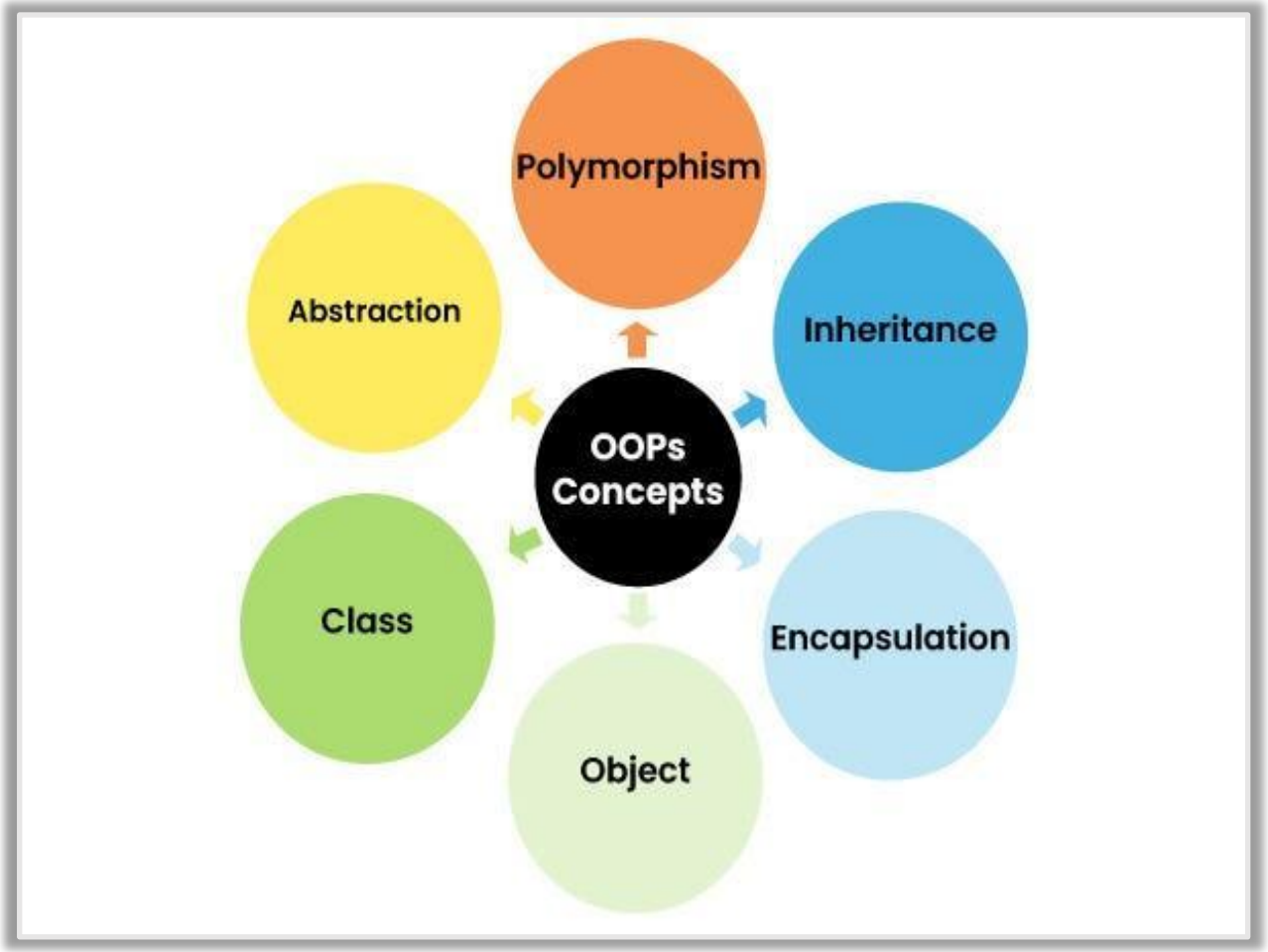
Drawbacks

- The **top-down approach** makes the program harder to maintain.
- It uses too many **global variables**, which increases memory usage and is not recommended.
- It focuses more on **processes (functions)** and does not give equal importance to **data**, so data is treated carelessly.
- Data can move **freely between functions**, without control.

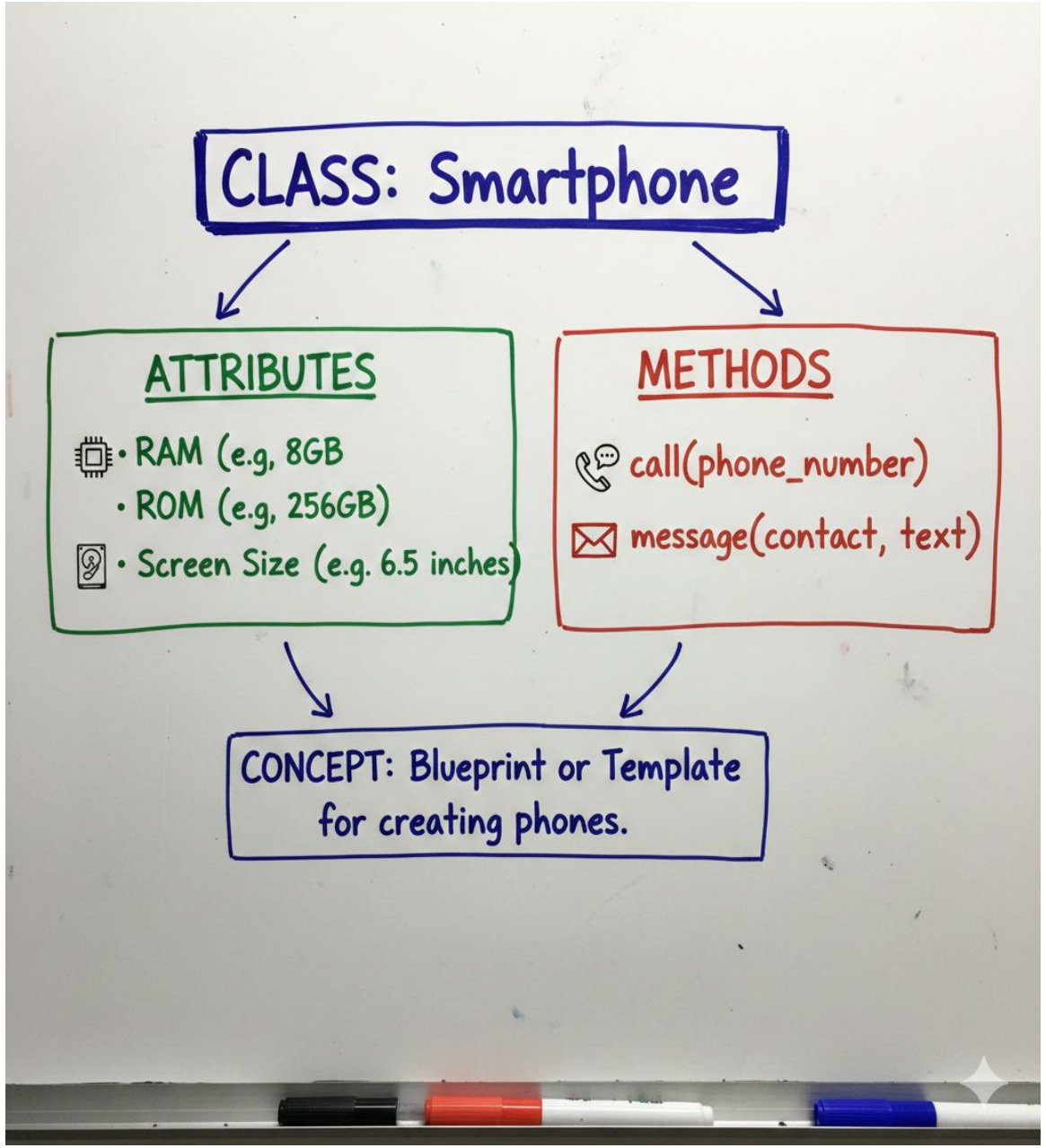
OOPs Introduction

- OOP is an abbreviation that stands for **Object-oriented programming** paradigm.
- It is defined as a programming model that uses the concept of **objects** which refers to real-world entities with state and behavior.
- In simple terms, OOPS is a programming style where everything is grouped into objects to make programs easier to build, understand, and reuse.
- Advantages of OOP
  - Provides a clear structure to programs
  - Makes code easier to maintain, reuse, and debug
  - Helps keep your code DRY (**Don't Repeat Yourself**)
  - Allows you to build reusable applications with less code

Principles of OOPs Concepts



1. **Class**
2. **Object**
3. **Encapsulation**
4. **Inheritance**
5. **Polymorphism**
6. **Abstraction**



What is a Class in Python?

- a **class** is a user defined entity (data types) that defines the type of data an object can contain and the actions it can perform.
- A class is a blueprint for creating objects.
- A **class** is like a **design or plan** that describes:
  - what data an object will have (**attributes**)
  - what actions an object can do (**methods**)
- It does **not** hold real data by itself — it only defines the structure.

What is Objects in python?

- An object is created based on that class.
- An **object** is a **real-world instance**
- holds **real data**
- can perform **actions**
- Every object has its own *separate* data.

Note:

Pass Statement

- The **pass** statement is a **do-nothing** statement.
- It is used when you want to **write a block of code**, but you **don't want to put any code inside it yet**.
- Python requires an indented block, so pass acts as a **placeholder**.

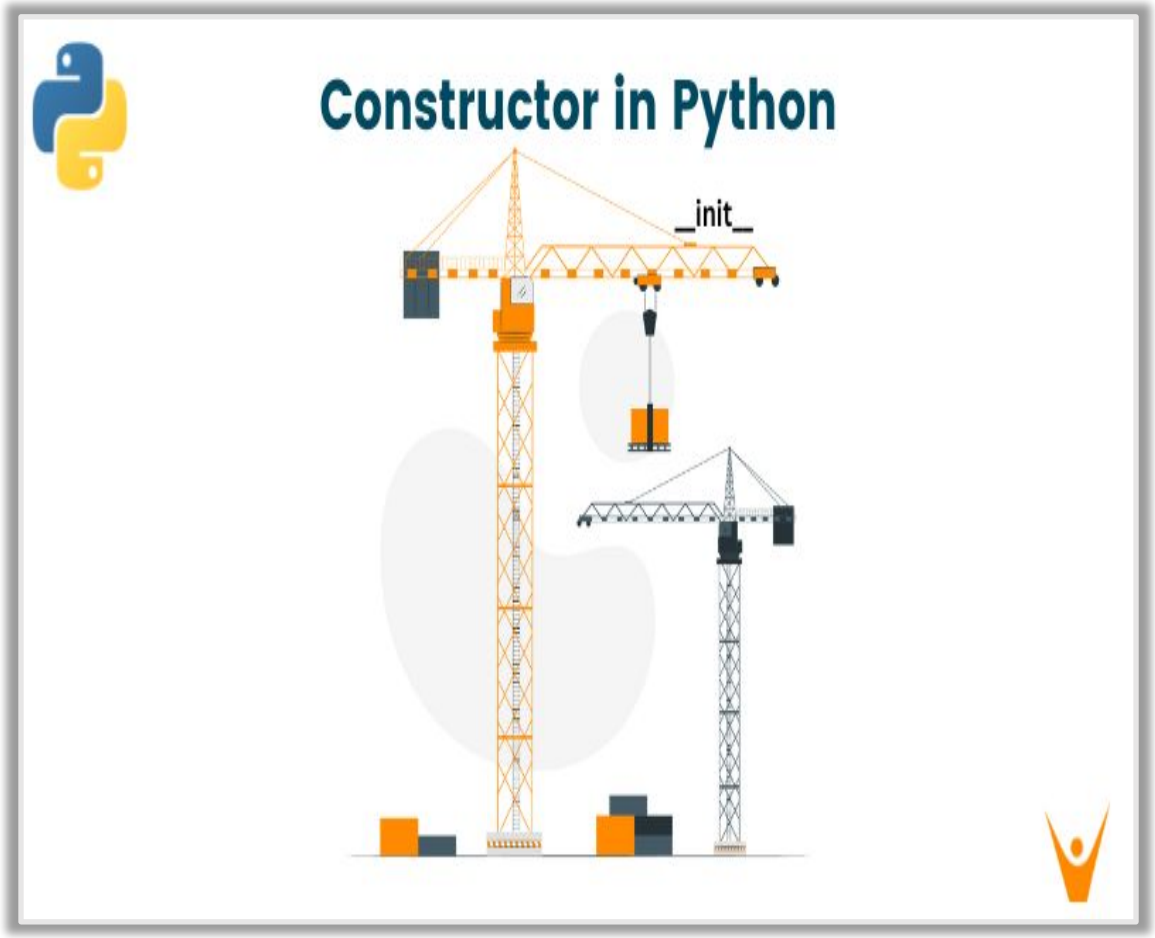
Self Parameter

- self refers to the **current object (instance)** of the class.
- It is used to access:
  - the object's **attributes**
  - the object's **methods**
- Without self, a method **cannot** use or change the object's data.

Types of Attributes

Feature	Class Attribute	Instance Attribute	Local Attribute
Where defined?	Inside class, **outside methods**	Inside methods using **self**	Inside a method **without self**
Belongs to	The **class** (shared by all objects)	Each **individual object**	Only to the **method** where it is created
Lifetime	As long as the class exists	As long as the object exists	Only during the method's execution
How many copies?	**One copy** shared by all objects	**Separate copy** for each object	Created and destroyed inside the method
Accessed using	ClassName.attribute or self.attribute	self.attribute	Variable name directly inside the method
Purpose	Store common data for all objects	Store object-specific data	Temporary calculations inside a method
Example	company = "Toyota"	self.name = "Aju"	x = 10

Constructors



- A **constructor** is a special method in a class that **runs automatically** when an object is created.
- used for instantiating an object.
- The task of constructors is to initialize(assign values) to the data members of the class when an object of the class is created.
- the __init__() method is called the constructor and is always called when an object is created.

Syntax of constructor declaration :

```
def __init__(self):
    # body of the constructor
```

Type of Constructors

1. Default Constructor

- A constructor **with no parameters**.
- It uses **default values** for attributes.

```
class Student:
    def __init__(self):
        self.name = "Unknown"
        self.age = 0

s1 = Student()
print(s1.name, s1.age)
```

2. Parameterized Constructor

- A constructor that **takes parameters**.
- Allows you to **pass values** while creating the object.

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

s1 = Student("Aju", 18)
print(s1.name, s1.age)
```

Destructors

- Destructors are called when an object gets destroyed.
- destructors are not needed as much as in C++ because Python has a garbage collector that handles memory management automatically.
- The __del__() method is a known as a destructor method in Python.
- It is called when all references to the object have been deleted i.e when an object is garbage collected.

Syntax of destructor declaration :

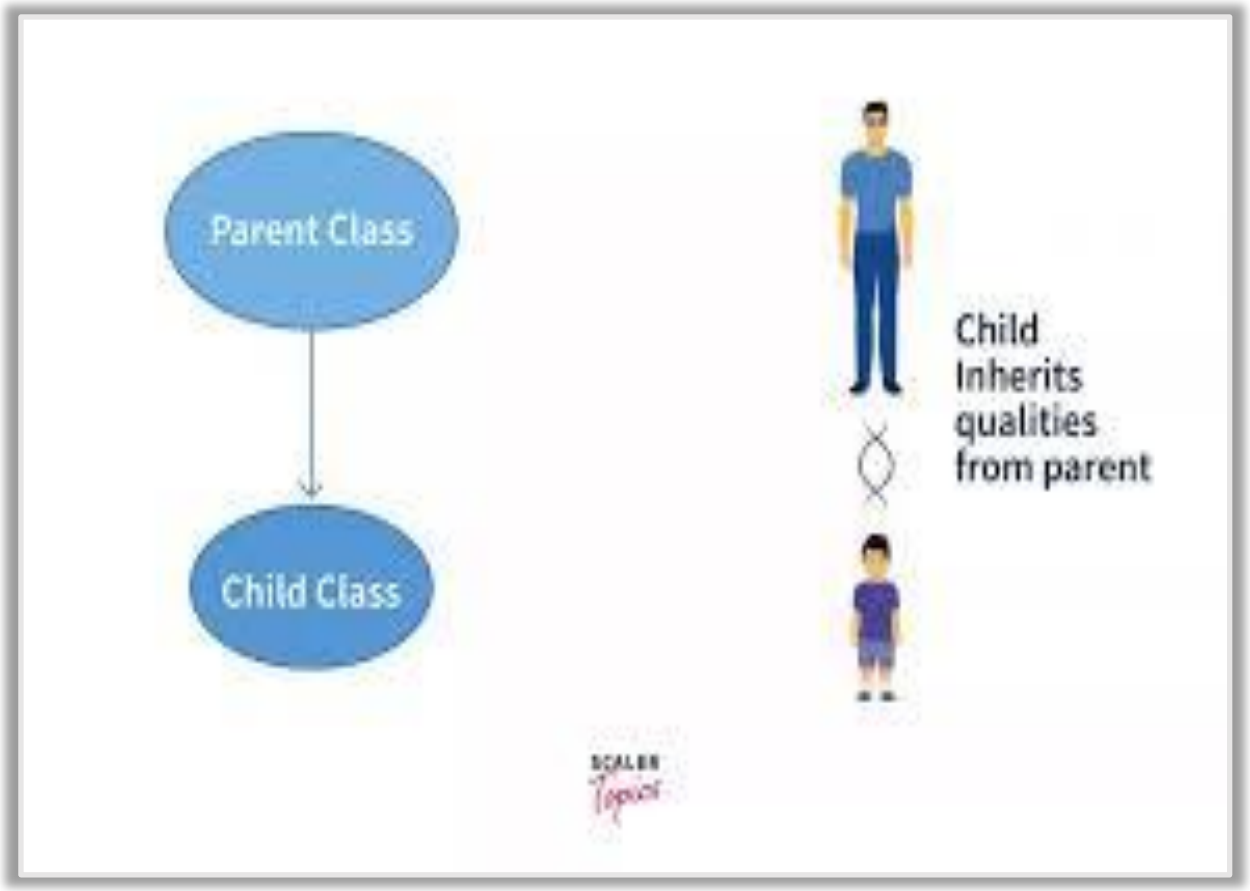
```
def __del__(self):
    # body of destructor
```

Accessing Attributes of Objects in Python

- **getattr(obj, name[, default])** − to access the attribute of object.
- **hasattr(obj,name)** − to check if an attribute exists or not.
- **setattr(obj,name,value)** − to set an attribute. If attribute does not exist, then it would be created.
- **delattr(obj, name)** − to delete an attribute.

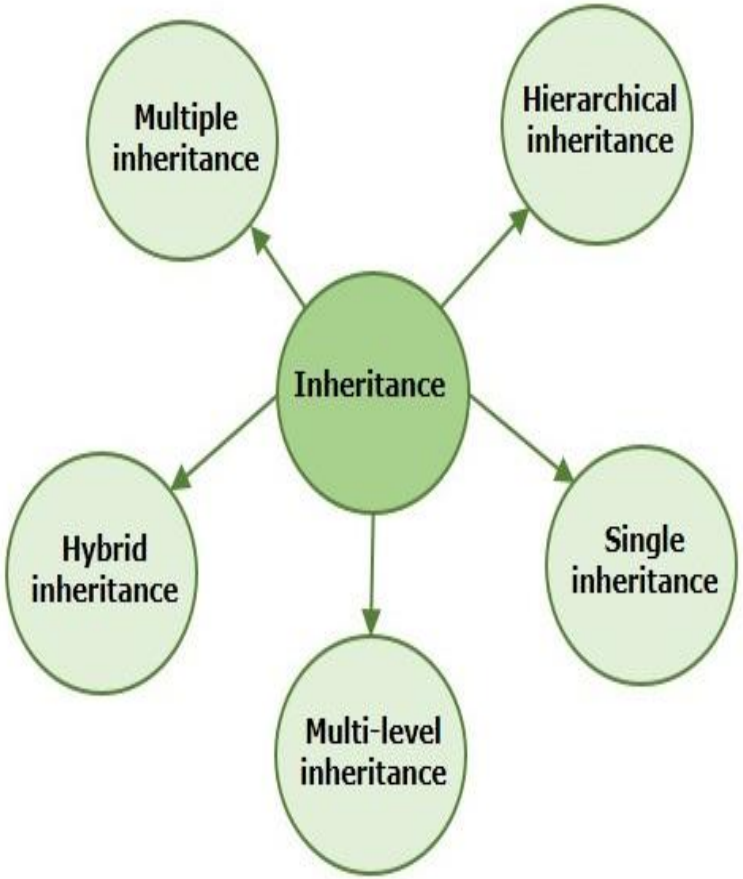
```
hasattr(emp1, 'age')

getattr(emp1, 'age')

setattr(emp1, 'age', 8)

delattr(empl, 'age')
```

Inheritance



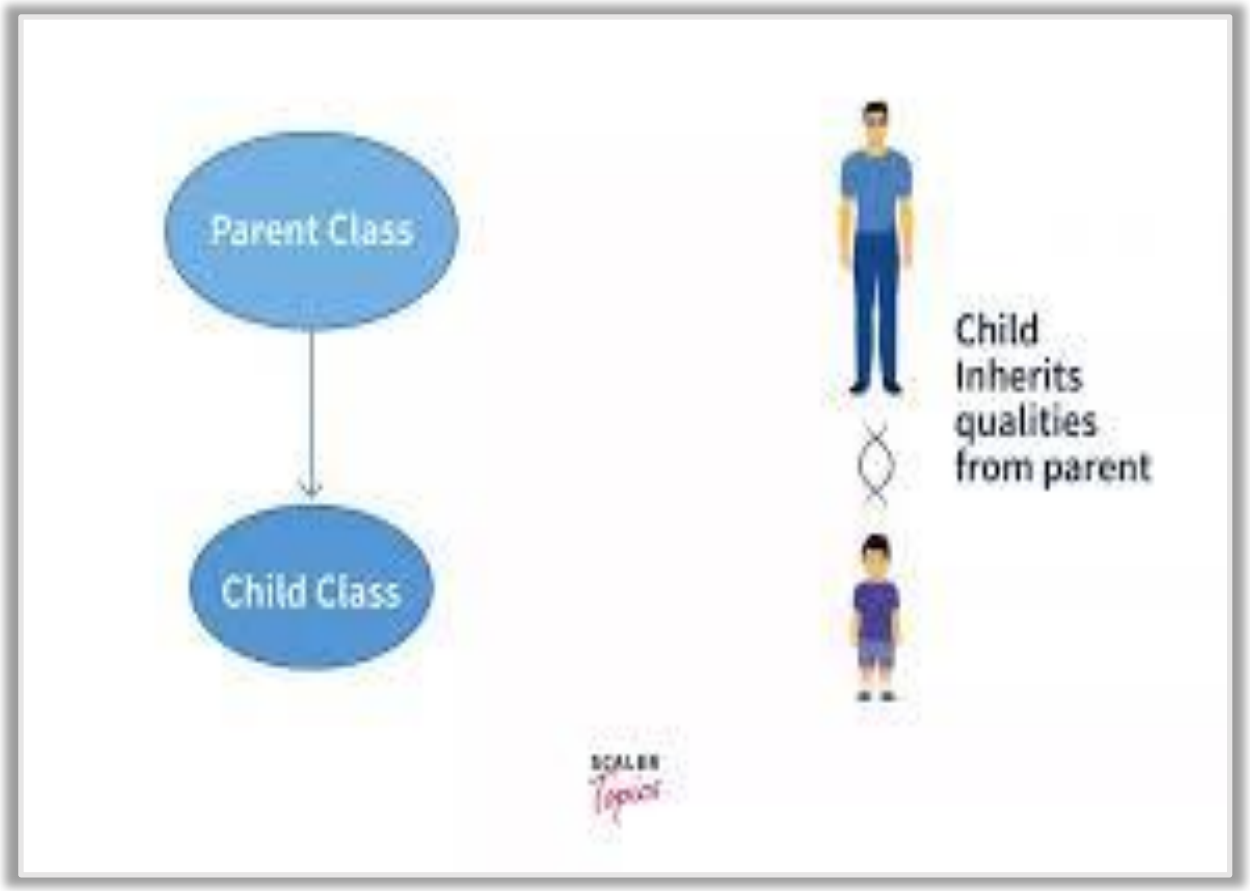
- Inheritance allows us to define a class that inherits all the methods and properties from another class.
- **Parent class** is the class being inherited from, also called base class.
- **Child class** is the class that inherits from another class, also called derived class.

Types of Inheritance



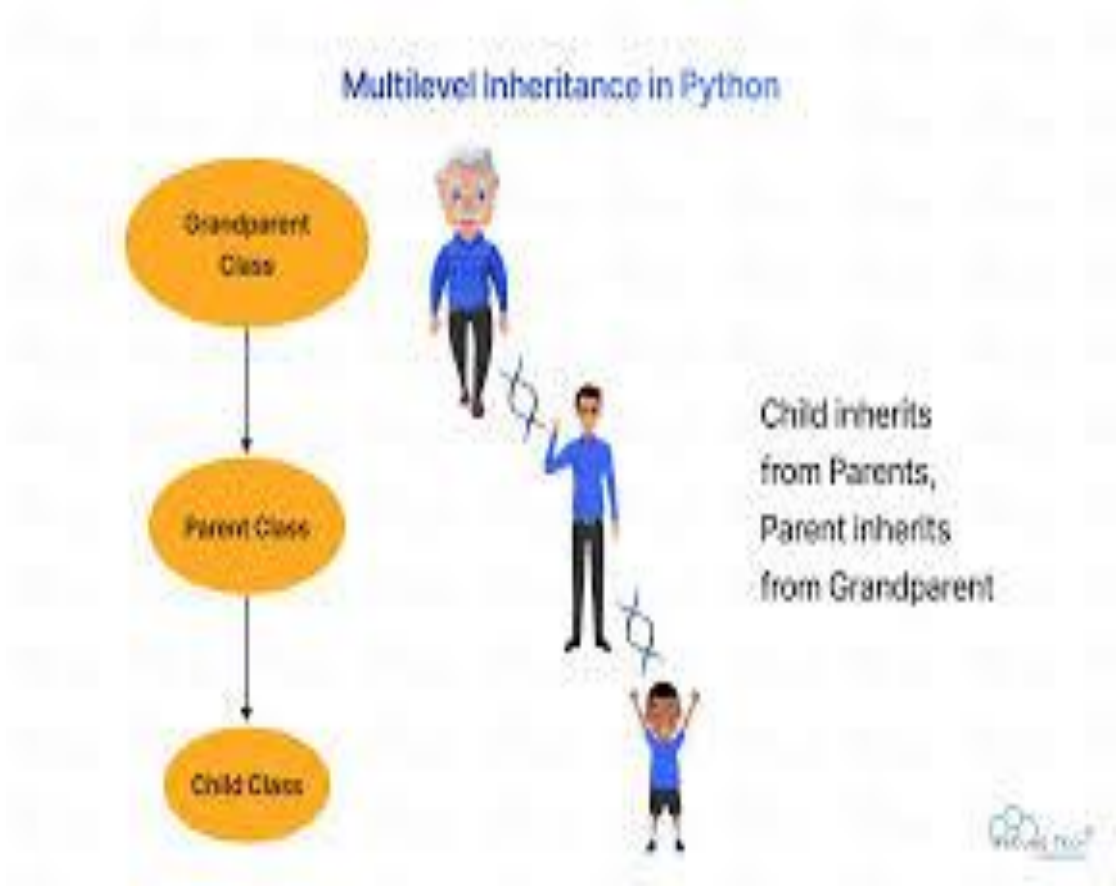
- Single Inheritance
- Multiple Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Hybrid Inheritance

Single Inheritance



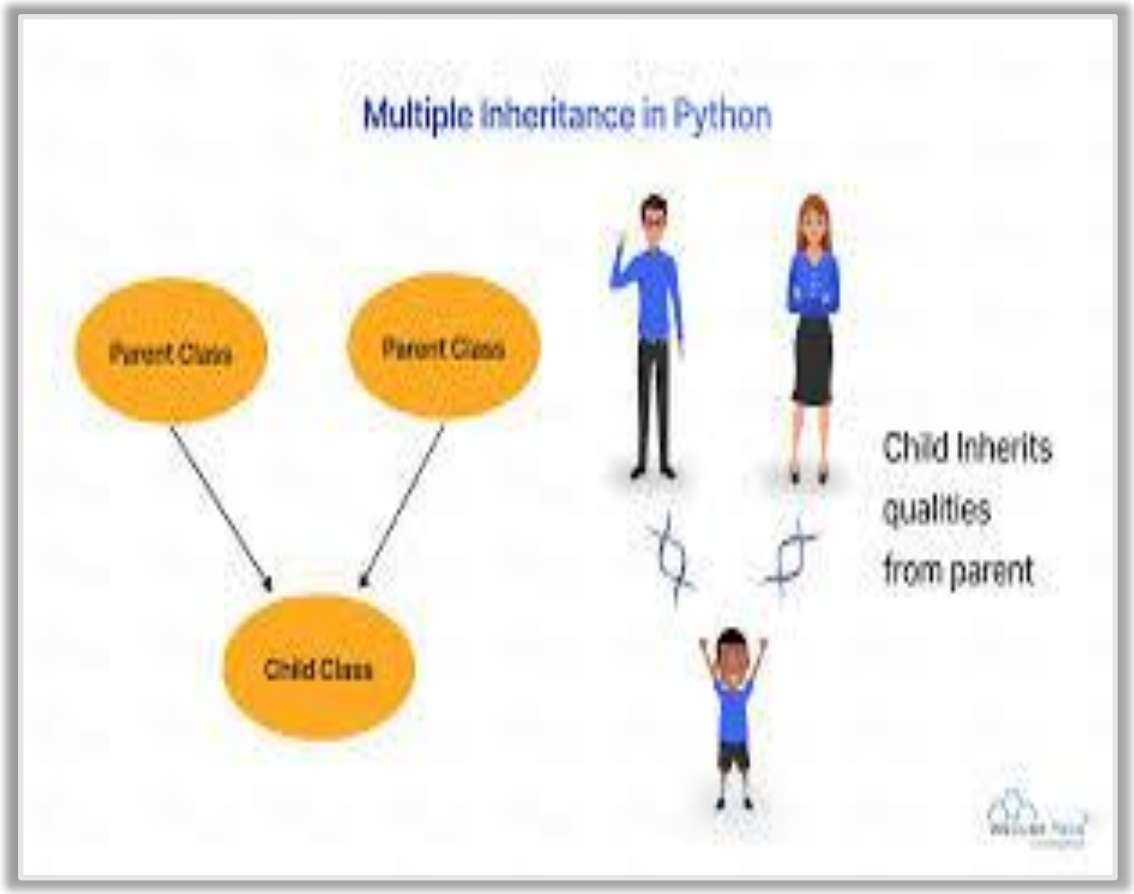
If a parent class is inherited by a single child class, then the relation between the parent and child classes is known as single inheritance.

Multi-level Inheritance



- The concept of inheriting the members from multiple classes into a single class is known as multi-level inheritance.
- In multi-level inheritance, the child class can be a parent class to another child class and the process goes on.
- Thus, multi-level inheritance can also be called multiple level inheritance.

Multiple Inheritance



- If a single child class inherits two or more classes, then the relation between the child class and the multiple parent classes is known as Multiple Inheritance.
- All the members of the classes that are being inherited are accessible by the child class.
- The parent classes that the child class inherits are declared in an order using parenthesis.
- **class** child class(parent class1, parent class2, parent class3), ...):

Method Resolution Order (MRO)

MRO tells Python which class's method should be executed first when more than one class has the same method name.

When a class inherits from **multiple parent classes**, Python must decide:

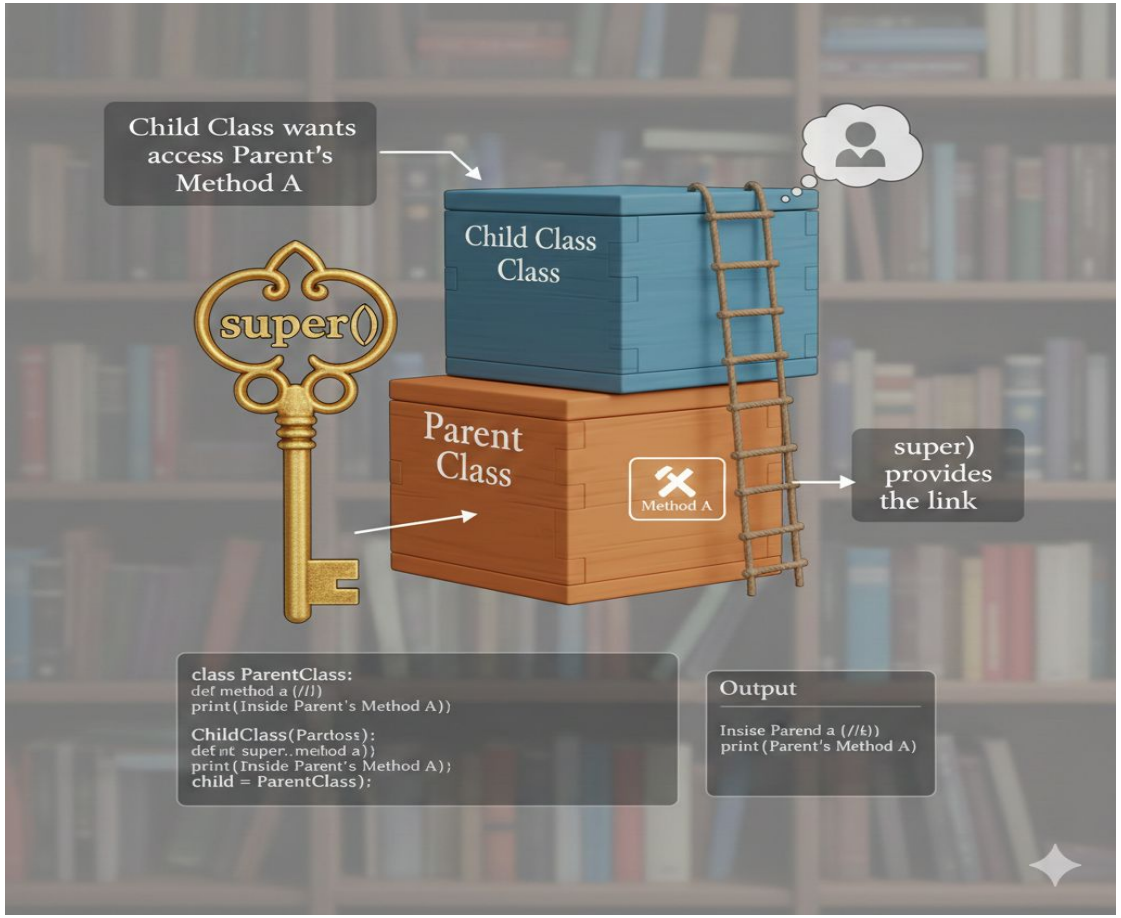
- From which class should it take the method?
- In what order should parent classes be searched?
- MRO solves this confusion.

Python uses an algorithm called **C3 Linearization**, which:

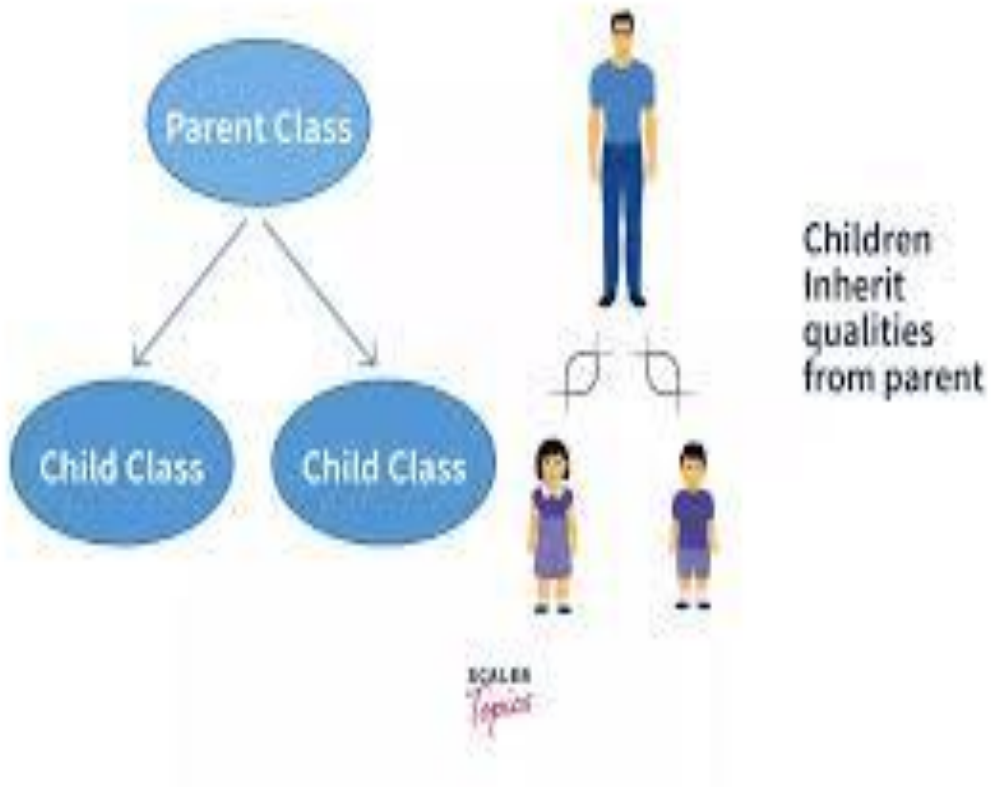
- Follows **left-to-right** order of inheritance
- Ensures **child class overrides parent methods**
- Avoids repeating the same class

Super( )

- **super()** is a built-in Python function used to **call methods of a parent (super) class** from a child (sub) class.
- **Why we use super()**
  - To **avoid code duplication**
  - To **access parent class methods**
  - To **support multiple inheritance properly** (works with MRO)

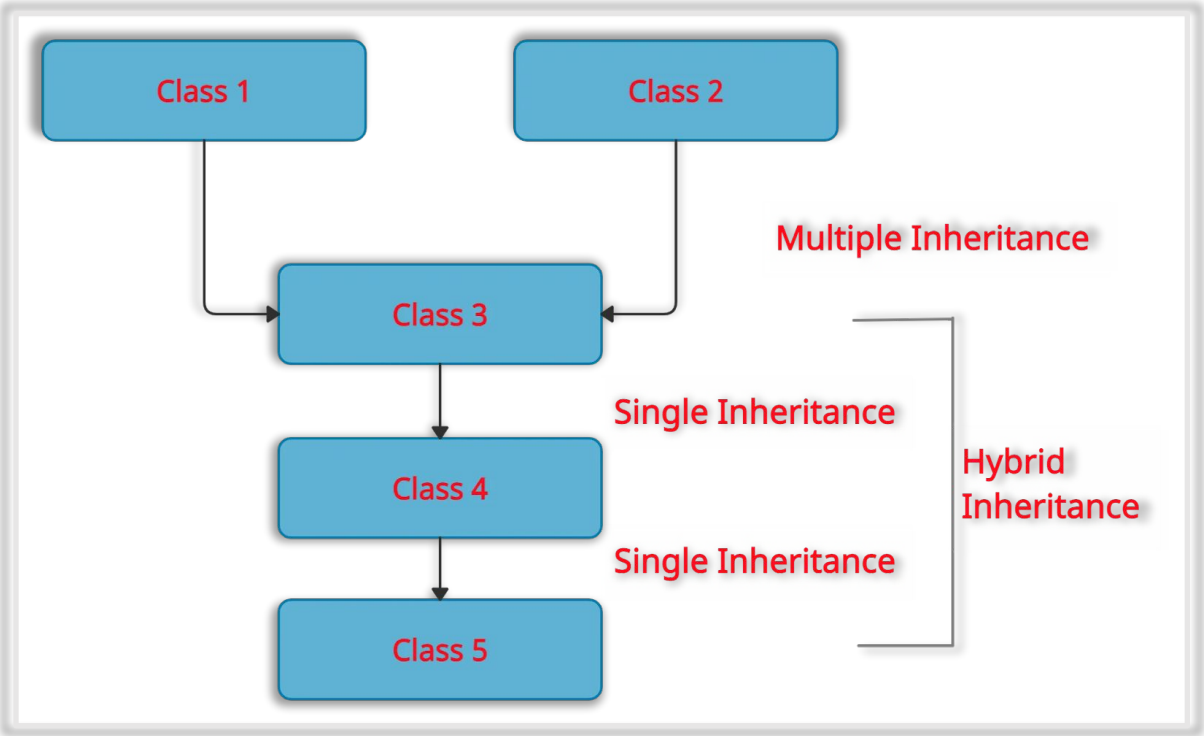


Hierarchical Inheritance



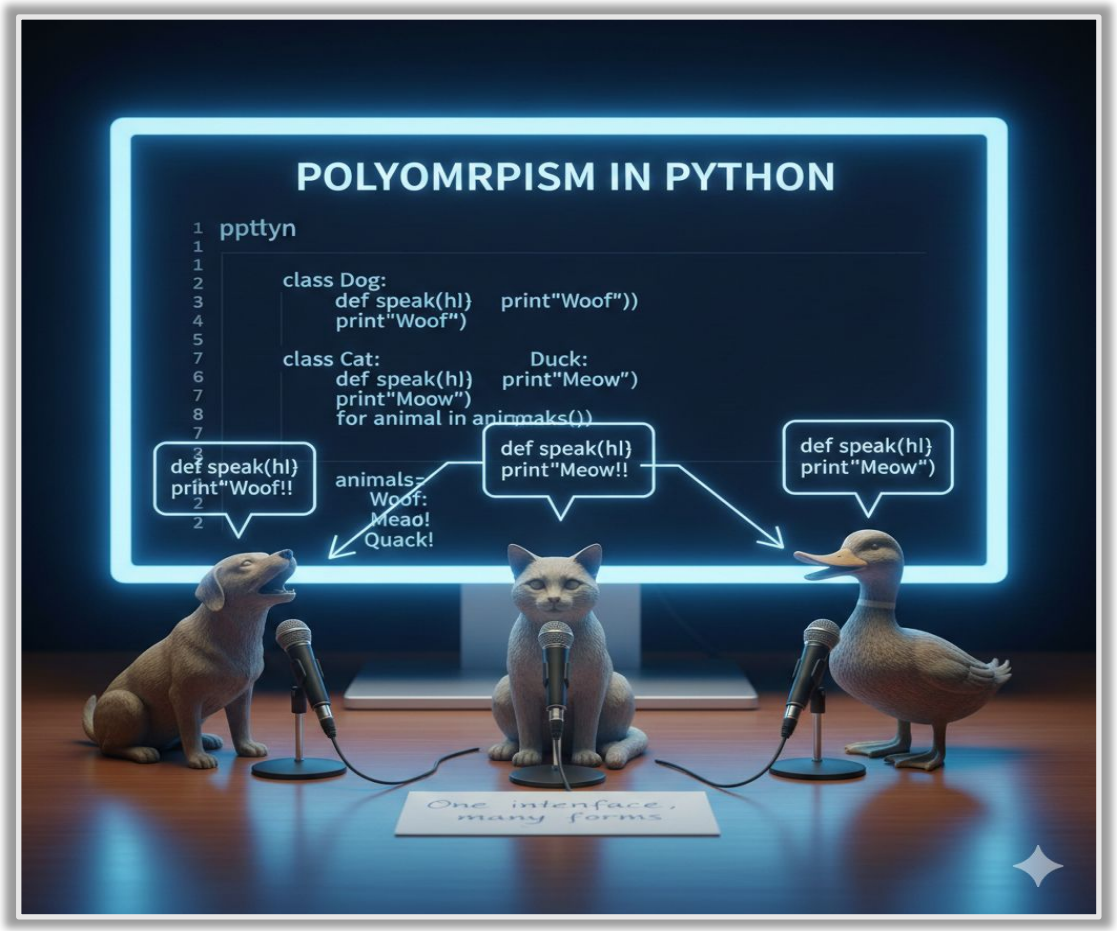
- In hierarchical inheritance, a single parent class will have multiple child classes which are at the same level and the members of the parent class are accessible to all the child classes.
- In this type of inheritance, the child1 members are not accessible by child2.

Hybrid Inheritance



- if a class has different types of inheritance implemented in it, then this inheritance is known as Hybrid inheritance.
- As the name itself suggests there will be two or more inheritances implemented.

Polymorphism



- Polymorphism means the ability to represent an object in many forms using a single interface.
- Polymorphism is derived from two different words poly which means many, morphs means many forms.
- Polymorphism in Python allows us to use the operators, methods, constructors in different forms and perform overloading and overriding on them.
- The different types of polymorphism are:-
  - Overloading
  - Overriding

Overloading in python

- Based on the values we pass if an object changes its behavior and returns a different outcome then it is known as overloading.
- Thus we can use the same operator or method for different purposes.
- Overloading in Python is of three types:-
  - Operator Overloading
  - Method Overloading
  - Constructor Overloading

Operator Overloading in python

- If we can use the same operator for multiple purposes then it is known as Operator Overloading.
- Python does support Operator Overloading.
- we can perform operator overloading on the “+” operator.
- If we pass two numbers then the “+” operator returns the result of addition performed between the numbers.
- The same operator if we pass two strings then it concatenates them and returns the result as a single string.
- if we pass the multiplication operator(*) between two numbers then it returns the product of the multiplication.
- The same ” * ” operator if we pass it between an integer and a string then it performs the string repetition operation.
- Thus the same operators performing different operators based on the values we pass is called Operator Overloading.

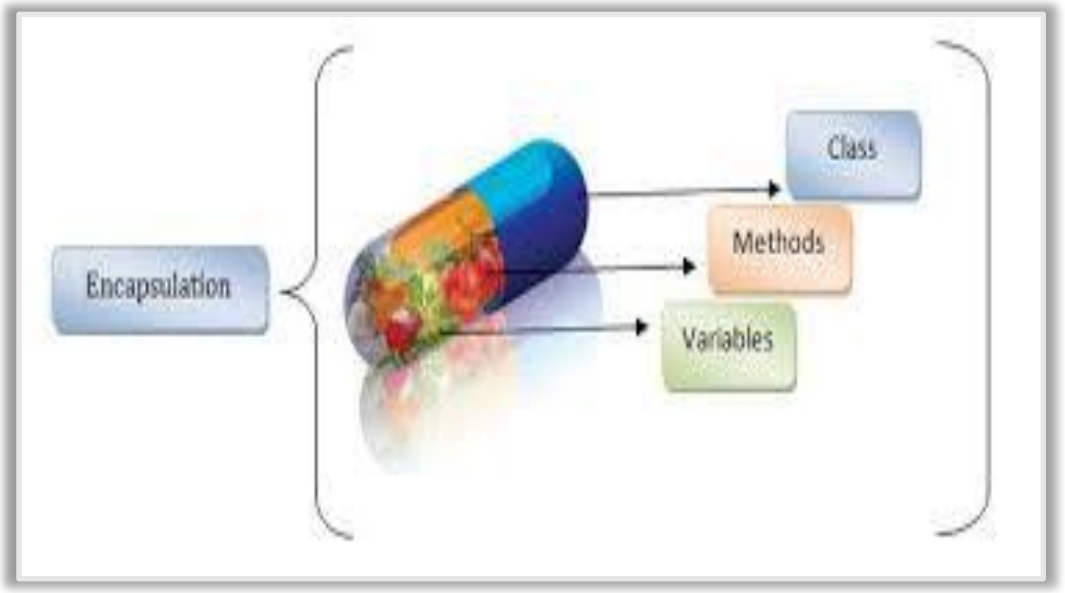
Method Overloading in python

- If there are two or more methods in a class with the same name but a different number of arguments then it is known as Method overloading. In Python, Method Overloading is not supported.
- Even though method overloading is not supported in Python, if we try to declare multiple methods with the same name but different arguments then the bottom-most or last method is considered and executed.

Method Overriding in python

- To override a method in the parent class we need to redefine the same method in the child class. So when the method is called from the object of the child class the new method will be called due to Method Resolution Order(MRO).
- class A is inherited by class B. But in class B we are overriding the method() by redefining it. Now if we call the method(), the child class method is called because it is overriding the method in the parent class.
- If we still want to use the parent class method then we can use super() function to load the method into child class and make use of both the methods.

Encapsulation



- Encapsulation means **wrapping data (variables) and methods (functions)** into a single unit called a **class**, and **protecting** that data from being accessed directly.
- It allows you to **control access** to the data using:
- public
- protected
- private attributes
- getter and setter methods

Python - Access Modifiers

- The **Python access modifiers** are used to restrict access to class members (i.e., variables and methods) from outside the class.
- **Public members** − A class member is said to be public if it can be accessed from anywhere in the program.
- **Protected members** − They are accessible from within the class as well as by classes derived from that class.
- **Private members** − They can be accessed from within the class only.

Abstraction

- **Abstraction** is an **object-oriented programming (OOP) concept** that **hides implementation details** and **shows only essential features** to the user.
- **Why abstraction is needed**
  - Reduces **complexity**
  - Improves **security**
  - Makes code **easy to use and maintain**
  - Allows focus on **important features only**